

分支任务lab5: gdb 调试系统调用以及返回

一、准备工作

首先同样开启三个终端

终端1 启动qemu模拟器

make debug

终端2 附加qemu进程

pgrep -f qemu-system-riscv64

sudo gdb

(gdb) attach <PID>

(gdb) handle SIGPIPE nostop noprint

(gdb) c

终端3 调试内核

make gdb

二、观察ecall指令和sret指令是如何被qemu处理

要求：在大模型的帮助下，完成整个调试的流程，观察一下ecall指令和sret指令是如何被qemu处理的，并简单阅读一下调试中涉及到的qemu源码，解释其中的关键流程。

2.1 设置syscall(ecall)相关断点

在syscall处打断点

```
(gdb) b syscall
Breakpoint 2 at 0xffffffffc0205144: file kern/syscall/syscall.c, line 83.
(gdb) b user/libs/syscall.c:26
No source file named user/libs/syscall.c.
< on future shared library load? (y or [n]) y
Breakpoint 3 (user/libs/syscall.c:26) pending.
(gdb)
```

代码块

```
1 (gdb) b syscall
2 Breakpoint 2 at 0xffffffffc0205144: file kern/syscall/syscall.c, line 83.
3 (gdb) b user/libs/syscall.c:26
4 No source file named user/libs/syscall.c.
5 < on future shared library load? (y or [n]) y
```

```
6 Breakpoint 3 (user/libs/syscall.c:26) pending.
7 (gdb)
```

发现gdb报“No source file named user/libs/syscall.c”，因为当前gdb会话里还没有加载包含该源文件调试符号的目标文件（symbol table）。我们已经用gdb载入并连接了内核二进制

（bin/kernel），内核符号可用，所以在内核源码上下断点成功；但syscall.c属于用户态程序的源码，它的符号不在当前symbol table中，因此gdb找不到对应源文件和行号，所以把断点设为“pending”（将来若加载到符号时再激活）。

为了能够成功执行b user/libs/syscall.c:26，在ecall附近设置断点，我们需要把用户程序的符号加载到当前gdb，在gdb中用add-symbol-file把用户程序ELF的符号表映射到其实际加载地址，这样gdb就知道对应源文件，能按文件:行设置断点。

add-symbol-file后面应该加上“已经被编译好的程序/模块”的符号表，我们找一个包含syscall.c源文件的符号表文件即可，这里我们选择obj/__user_exit.out，执行如下

```
10.
(gdb) add-symbol-file obj/__user_exit.out
add symbol table from file "obj/__user_exit.out"
(y or n) y
Reading symbols from obj/__user_exit.out...
(gdb) break user/libs/syscall.c:18
Breakpoint 2 at 0x800082: file user/libs/syscall.c, line 19.
(gdb)
```

代码块

```
1 (gdb) b kern_entry
2 (gdb) add-symbol-file obj/__user_exit.out
3 add symbol table from file "obj/__user_exit.out"
4 (y or n) y
5 Reading symbols from obj/__user_exit.out...
6 (gdb) break user/libs/syscall.c:18
7 Breakpoint 1 at 0x8000f8: file user/libs/syscall.c, line 19.
```

然后我们输入continue继续

```
(gdb) c
Continuing.

Breakpoint 2, syscall (num=2) at user/libs/syscall.c:19
19          asm volatile (
```

可以看到，断点生效了，代码停在了syscall函数中的内联汇编代码开头处，接着我们输入si单步执行汇编指令，如下所示

```
Breakpoint 2, syscall (num=2) at user/libs/syscall.c:19
19          asm volatile (
(gdb) si
Warning:
Cannot insert breakpoint 1: Cannot access memory at address 0xffffffff
ffc0200000

Command aborted.
(gdb) delete 1
(gdb) si
0x0000000000800084          19          asm volatile (
(gdb)
```

可以看到这里显示无法访问地址 `0xffffffffffc020000`，导致断点 1 插入失败，由于 `0xffffffffffc020000` 是内核地址，而当前调试上下文是用户态程序，权限不足，默认无法访问该内存区域，导致断点插入失败，由于这个断点对我们这部分整个调试影响不大，我们这里直接删除第一个断点即可。

但我们这里不是使用 `si` 进行单步调试吗，不涉及断点 1，为什么这里还会报错呢？

大模型给出的回答是（整理版）：

当我们执行 `si` 单步时，GDB 会先遍历所有已注册的断点（包括断点 1），尝试将断点指令“插入”到对应地址的内存中——但用户态无法写入 / 访问内核地址，因此触发“Cannot access memory at address 0xffffffffc020000”错误，所以这里本质是断点 1 的预处理失败，而非单步操作本身报错。

接着我们正常使用 `si` 进行单步调试，结果如下

```
(gdb) si
0x0000000000800084          19          asm volatile (
(gdb) x/7i $pc
=> 0x800084 <syscall+34>:      ld      a1,40(sp)
0x800086 <syscall+36>:      ld      a2,48(sp)
0x800088 <syscall+38>:      ld      a3,56(sp)
0x80008a <syscall+40>:      ld      a4,64(sp)
0x80008c <syscall+42>:      ld      a5,72(sp)
0x80008e <syscall+44>:      ecall
0x800092 <syscall+48>:      sd      a0,28(sp)
(gdb)
```

我们还可以使用命令 `x/7i $pc` 从当前 PC 位置开始，反汇编显示 7 条指令，这与源码中的指令几乎是一致的

源码

```
1  asm volatile (
2      "ld a0, %1\n"
3      "ld a1, %2\n"
```

```

4      "ld a2, %3\n"
5      "ld a3, %4\n"
6      "ld a4, %5\n"
7      "ld a5, %6\n"
8      "ecall\n"
9      "sd a0, %0"

```

其中我们可以看到ecall指令对应的位置在0x80008e <syscall+44>，命令i r \$pc可以查看当前pc的值为0x800086 <syscall+34>，于是我们再多运行几次si，使得pc值变为0x80008e <syscall+44>，即停在ecall指令的上一条指令的位置，结果如下所示

```

0x80008a <syscall+40>:      ld      a4,64(sp)
0x80008c <syscall+42>:      ld      a5,72(sp)
0x80008e <syscall+44>:      ecall
0x800092 <syscall+48>:      sd      a0,28(sp)
(gdb) i r $pc
pc                0x800084 0x800084 <syscall+34>
(gdb) si
0x0000000000800086      19      asm volatile (
(gdb) si
0x0000000000800088      19      asm volatile (
(gdb) si
0x000000000080008a      19      asm volatile (
(gdb) si
0x000000000080008c      19      asm volatile (
(gdb) si
0x000000000080008e      19      asm volatile (
(gdb) i r $pc
pc                0x80008e 0x80008e <syscall+44>
(gdb)

```

可以看到现在停在了ecall指令的前一条指令的位置，接下来去为qemu设置相关断点，再回来继续运行

2.2 为qemu设置ecall相关断点

在大模型的帮助下，我们首先使用命令

```
cd /home/osaq/qemu-4.1.1-debug
```

```
nm -C /home/osaq/qemu-4.1.1-debug/riscv64-softmmu/qemu-system-riscv64 2>/dev/null |
egrep -i 'translate|ecall|exception|raise|helper|tlb'
```

得到了我们的qemu中可能与处理ecall相关的一系列函数，部分如下图所示

```

osaq@LAPTOP-QS28APKV:~$ cd /home/osaq/qemu-4.1.1-debug
osaq@LAPTOP-QS28APKV:~/qemu-4.1.1-debug$ nm -C /home/osaq/qemu-4.1.1-debug/riscv64-softmmu/qemu-system-riscv64 2>/dev/nu
ll | egrep -i 'translate|ecall|exception|raise|helper|tlb'
0000000000d0b3a0 D A64TLBI_TLBIMapper
0000000000d0b1a0 d TLBIPairs
0000000000f05d66 B _TRACE_AHCI_IRQ_RAISE_DSTATE
0000000000eb6be0 D _TRACE_AHCI_IRQ_RAISE_EVENT
0000000000f061d4 B _TRACE_ESP_RAISE_IRQ_DSTATE
0000000000ebc1a0 D _TRACE_ESP_RAISE_IRQ_EVENT
0000000000f0617a B _TRACE_MEGASAS_IRQ_RAISE_DSTATE
0000000000ebbc00 D _TRACE_MEGASAS_IRQ_RAISE_EVENT
0000000000f06174 B _TRACE_MEGASAS_MSIX_RAISE_DSTATE
0000000000ebbba0 D _TRACE_MEGASAS_MSIX_RAISE_EVENT
0000000000f06176 B _TRACE_MEGASAS_MSI_RAISE_DSTATE
0000000000ebbbc0 D _TRACE_MEGASAS_MSI_RAISE_EVENT
0000000000f05e34 B _TRACE_MILKYMIST_MINIMAC2_RAISE_IRQ_RX_DSTATE
0000000000eb7d20 D _TRACE_MILKYMIST_MINIMAC2_RAISE_IRQ_RX_EVENT
0000000000f05cca B _TRACE_MILKYMIST_UART_RAISE_IRQ_DSTATE
0000000000eb6040 D _TRACE_MILKYMIST_UART_RAISE_IRQ_EVENT
0000000000f063ae B _TRACE_TPM_TIS_RAISE_IRQ_DSTATE
0000000000eba80 D _TRACE_TPM_TIS_RAISE_IRQ_EVENT
0000000000f058ee B _TRACE_TRANSLATE_BLOCK_DSTATE
0000000000eb12a0 D _TRACE_TRANSLATE_BLOCK_EVENT
0000000000f066c4 B _TRACE_VHOST_IOTLB_MISS_DSTATE
0000000000ec2940 D _TRACE_VHOST_IOTLB_MISS_EVENT
00000000004844a4 t _nocheck_trace_ahci_irq_raise
0000000000522d09 t _nocheck_trace_esp_raise_irq
0000000000517575 t _nocheck_trace_megasas_irq_raise
0000000000517404 t _nocheck_trace_megasas_msi_raise

```

由于数量非常多，我们将得到的所有函数交给大模型，让大模型帮我们筛选出ecall相关的重要的函数，结果主要是trans_ecall，riscv_raise_exception，riscv_cpu_do_interrupt、riscv_cpu_set_mode和cpu_handle_interrupt这五个函数

QEMU 处理 ecall（用户态 U → 内核态 S）流程主要为：

- trans_ecall()：**在 TCG 翻译阶段识别到 guest 的 ecall 指令，并生成“抛出异常”的 TCG 中间代码，而不是立即执行异常。这是 **翻译期**，还没有发生特权级切换，只是告诉 TCG 这条指令会导致异常。
- raise_exception()：**运行期执行到这条 TB 时，TCG 生成的 host 代码调用异常 helper，正式触发异常。这是 **运行期**，从“普通 guest 指令执行”转向“异常处理流程”，真正的异常是在这里才发生的。
- riscv_raise_exception()：**设置当前 CPU 的异常编号，并通过 cpu_loop_exit_restore() 强制退出当前 TB。这里不做 CSR 更新，只负责 **“告诉 QEMU：有异常了”**。
- cpu_exec() loop：**QEMU 的主执行循环捕获异常跳转，进入统一的异常处理框架。控制权从 TB 回到 QEMU 主循环
- cpu_handle_exception()：**根据异常类型与 CPU 架构，分发到具体架构相关的**异常处理**函数。
- riscv_cpu_do_interrupt()：**按 RISC-V 特权规范处理异常：判断委托、选择目标特权级，并准备更新 CSR。这里会 riscv_cpu_set_mode() 切换当前 CPU 特权级。它决定是进 **S-mode 还是 M-mode**，是 **软件模拟的“硬件陷入逻辑”**

接下来我们选取五个重要的函数为qemu加断点，首先我们使用ctrl+c中断当前的运行，然后输入

(gdb) b trans_ecall

(gdb) b riscv_raise_exception

(gdb) b cpu_handle_interrupt

(gdb) b riscv_cpu_do_interrupt

(gdb) b riscv_cpu_set_mode

最后我们再输入c(continue)，继续运行

```
Continuing.
^C
Thread 1 "qemu-system-ris" received signal SIGINT, Interrupt.
0x000073af14754cb6 in __ppoll (fds=0x559428211410, nfd=6,
    timeout=<optimized out>, sigmask=0x0)
    at ../sysdeps/unix/sysv/linux/ppoll.c:44
44      in ../sysdeps/unix/sysv/linux/ppoll.c
(gdb) b trans_ecall
Breakpoint 1 at 0x55941b144190: file /home/osaq/qemu-4.1.1-debug/ta
rget/riscv/insn_trans/trans_privileged.inc.c, line 22.
(gdb) b riscv_raise_exception
Breakpoint 2 at 0x55941b14690a: file /home/osaq/qemu-4.1.1-debug/ta
rget/riscv/op_helper.c, line 30.
(gdb) b cpu_handle_interrupt
Breakpoint 3 at 0x55941b0b58e0: file /home/osaq/qemu-4.1.1-debug/ac
cel/tcg/cpu-exec.c, line 523.
(gdb) b riscv_cpu_do_interrupt
Breakpoint 4 at 0x55941b14856a: file /home/osaq/qemu-4.1.1-debug/ta
rget/riscv/cpu_helper.c, line 504.
(gdb) b riscv_cpu_set_mode
Breakpoint 5 at 0x55941b147683: file /home/osaq/qemu-4.1.1-debug/ta
rget/riscv/cpu_helper.c, line 126.
(gdb) c
Continuing.
```

2.3 qemu处理ecall调试过程

接下来，我们将焦点再回到终端3（调试内核的gdb），我们输入c继续运行，可以看到终端2（调试qemu的gdb）立刻捕获到了一个中断，如下图所示

```
etext 0xc0205694 (virtual)
edata 0xc02a6250 (virtual)
end 0xc02aa6f4 (virtual)
Kernel executable memory footprint: 682KB
memory management: default_pmm_manager
physcial memory map:
  memory: 0x08000000, [0x80000000, 0x87ffffff].
vapaofset is 18446744070488326144
check_alloc_page() succeeded!
check_pgdir() succeeded!
check_boot_pgdir() succeeded!
use SLOB allocator
kmmalloc_init() succeeded!
check_vma_struct() succeeded!
check_vmm() succeeded.
++ setup timer interrupts
kernel_execve: pid = 2, name = "forktest".
Breakpoint

(gdb) si
0x0000000000000086 19 asm volatile (
(gdb) si
0x0000000000000088 19 asm volatile (
(gdb) si
0x000000000000008a 19 asm volatile (
(gdb) i r pc
pc 0x80008a 0x80008a <syscall+40>
(gdb) si
0x000000000000008c 19 asm volatile (
(gdb) i r pc
pc 0x80008c 0x80008c <syscall+42>
(gdb) si
0x000000000000008e 19 asm volatile (
(gdb) i r pc
pc 0x80008e 0x80008e <syscall+44>
(gdb) c
Continuing.
```

```
timeout=<optimized out>, sigmask=0x0)
    at ../sysdeps/unix/sysv/linux/ppoll.c:44
44      in ../sysdeps/unix/sysv/linux/ppoll.c: No such file or directo
ry.
(gdb) handle SIGPIPE nostop noprint
Signal Stop Print Pass to program Description
SIGPIPE No No Yes Broken pipe
(gdb) c
Continuing.
^C
Thread 1 "qemu-system-ris" received signal SIGINT, Interrupt.
0x00007bb925d5fcb6 in __ppoll (fds=0x573f6b26a410, nfd=6,
    timeout=<optimized out>, sigmask=0x0)
    at ../sysdeps/unix/sysv/linux/ppoll.c:44
44      in ../sysdeps/unix/sysv/linux/ppoll.c
(gdb) b trans_ecall
Breakpoint 1 at 0x573f55246190: file /home/osaq/qemu-4.1.1-debug/ta
rget/riscv/insn_trans/trans_privileged.inc.c, line 22.
(gdb) b riscv_raise_exception
Breakpoint 2 at 0x573f5524890a: file /home/osaq/qemu-4.1.1-debug/ta
rget/riscv/op_helper.c, line 30.
(gdb) b cpu_handle_interrupt
Breakpoint 3 at 0x573f551b78e0: file /home/osaq/qemu-4.1.1-debug/ac
cel/tcg/cpu-exec.c, line 523.
(gdb) b riscv_cpu_do_interrupt
Breakpoint 4 at 0x573f5524a56a: file /home/osaq/qemu-4.1.1-debug/ta
rget/riscv/cpu_helper.c, line 504.
(gdb) b riscv_cpu_set_mode
Breakpoint 5 at 0x573f55249683: file /home/osaq/qemu-4.1.1-debug/ta
rget/riscv/cpu_helper.c, line 126.
(gdb) c
Continuing.
[Switching to Thread 0x7bb92534d700 (LWP 47118)]

Thread 3 "qemu-system-ris" hit Breakpoint 3, cpu_handle_interrupt
(cpu=0x573f6b21e590, last_tb=0x7bb92534c9b8)
    at /home/osaq/qemu-4.1.1-debug/accel/tcg/cpu-exec.c:523
523      CPUClass *cc = CPU_GET_CLASS(cpu);
(gdb)
```



```
(gdb) c
Continuing.
[Switching to Thread 0x7bb92534d700 (LWP 47118)]

Thread 3 "qemu-system-ris" hit Breakpoint 3, cpu_handle_interrupt
(cpu=0x573f6b21e590, last_tb=0x7bb92534c9b8)
at /home/osaq/qemu-4.1.1-debug/accel/tcg/cpu-exec.c:523
523         CPUClass *cc = CPU_GET_CLASS(cpu);
(gdb)
```

可以看到，首先在b `cpu_handle_interrupt`函数处发生了中断

按我们上面的分析不是应该先调用`trans_ecall`函数吗，为什么首先中断在了`cpu_handle_interrupt`函数处？

大模型给出的回答（整理版）：

因为 `cpu_handle_interrupt()` 是 QEMU 在“每次执行 TB 之前”都会优先检查的统一入口，它处理的是“异步中断”，而不是 `ecall` 这种同步异常。它并不表示 `ecall` 已经发生，只是 QEMU 的主循环在“例行检查有没有外部/定时/软件中断”，每个 TB 前都会被调用它，`ecall` 是同步异常，只有在 TB 执行时才触发，所以调 `ecall` 时，`cpu_handle_interrupt` 会先被命中。

接下来我们先尝试多按几次c，整体观察qemu处理一次`ecall`的大致函数调用流程

```
Thread 3 "qemu-system-ris" hit Breakpoint 3, cpu_handle_interrupt
(cpu=0x5594281c5590, last_tb=0x73af11d419b8)
at /home/osaq/qemu-4.1.1-debug/accel/tcg/cpu-exec.c:523
523         CPUClass *cc = CPU_GET_CLASS(cpu);
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 4, riscv_cpu_do_interrupt
(cs=0x73af11d41910)
at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:504
504     {
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 5, riscv_cpu_set_mode (
env=0x0, newpriv=4754218856)
at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:126
126     {
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 3, cpu_handle_interrupt
(cpu=0x5594281c5590, last_tb=0x73af11d419b8)
at /home/osaq/qemu-4.1.1-debug/accel/tcg/cpu-exec.c:523
523         CPUClass *cc = CPU_GET_CLASS(cpu);
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 3, cpu_handle_interrupt
(cpu=0x5594281c5590, last_tb=0x73af11d419b8)
at /home/osaq/qemu-4.1.1-debug/accel/tcg/cpu-exec.c:523
523         CPUClass *cc = CPU_GET_CLASS(cpu);
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 3, cpu_handle_interrupt
(cpu=0x5594281c5590, last_tb=0x73af11d419b8)
at /home/osaq/qemu-4.1.1-debug/accel/tcg/cpu-exec.c:523
```

可以发现，前前后后多次调用了riscv_cpu_set_mode、riscv_cpu_do_interrupt和bcpu_handle_interrupt，因为 QEMU 的 RISC-V 异常处理把 ecall 当作同步中断来处理，翻译期的trans_ecall() 只是生成抛出异常的代码，执行期真正触发异常时需要多次调用cpu_handle_interrupt() 和 riscv_cpu_do_interrupt() 来检查和处理 pending 异常，同时每次特权级切换或状态更新都会调用 riscv_cpu_set_mode()。

由于这里这些函数调用次数太多了，处理ecall核心主要是从trans_ecall函数开始的，因此我们这里先将除了trans_ecall之外的四个断点使能状态设置为disable，再运行c，到trans_ecall处中断后，再恢复它们的使能状态，结果如下

```
(gdb) disable 2
(gdb) disable 3
(gdb) disable 4
(gdb) disable 5
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 1, trans_ecall (
    ctx=0x7bb92534c800, a=0x7bb92534c700)
    at /home/osaq/qemu-4.1.1-debug/target/riscv/insn_trans/trans_privileged.inc.c:22
22     {
(gdb) enable 2
(gdb) enable
(gdb) enable 3
(gdb) enable 4
(gdb) enable 5
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 2, riscv_raise_exception
    (env=0x8000000, exception=0, pc=0)
    at /home/osaq/qemu-4.1.1-debug/target/riscv/op_helper.c:30
30     {
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 4, riscv_cpu_do_interrupt
    (cs=0x573f55d4bec0 <qemu_global_mutex>)
    at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:504
504     {
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 5, riscv_cpu_set_mode (
    env=0x0, newpriv=624216352)
    at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:126
126     {
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 3, cpu_handle_interrupt
    (cpu=0x573f6b21e590, last_tb=0x7bb92534c9b8)
    at /home/osaq/qemu-4.1.1-debug/accel/tcg/cpu-exec.c:523
523         CPUClass *cc = CPU_GET_CLASS(cpu);
(gdb)
```

可以看到，这里经历了一个我们打的五个断点函数都被调用的过程，进一步证明了我们的断点选择是正确的，qemu处理ecall的过程确实会调用这几个函数。

接下来我们进行单步调试，进一步分析这几个函数具体的内部处理逻辑

1、trans_ecall

我们在trans_ecall函数内部使用n进行单步调试,得到了该函数完整的处理过程，具体如下所示

```
Thread 3 "qemu-system-ris" hit Breakpoint 1, trans_ecall (
    ctx=0x77fc71280800, a=0x77fc71280700)
    at /home/osaq/qemu-4.1.1-debug/target/riscv/insn_trans/trans_privileged.inc.c:22
22     {
(gdb) n
24     generate_exception(ctx, RISCV_EXCP_U_ECALL);
(gdb) n
25     exit_tb(ctx); /* no chaining */
(gdb) n
26     ctx->base.is_jump = DISAS_NORETURN;
(gdb) n
27     return true;
(gdb) n
28 }
(gdb) n
decode_insn32 (ctx=0x77fc71280800, insn=115)
    at target/riscv/decode_insn32.inc.c:1614
1614     if (trans_ecall(ctx, &u.f_empty)) return true;
(gdb) n
1731 }
(gdb) n
decode_opc (ctx=0x77fc71280800)
    at /home/osaq/qemu-4.1.1-debug/target/riscv/translate.c:746
746     if (!decode_insn32(ctx, ctx->opcode)) {
(gdb) n
750 }
(gdb) n
riscv_tr_translate_insn (dcbase=0x77fc71280800,
    cpu=0x5c993b4d3590)
    at /home/osaq/qemu-4.1.1-debug/target/riscv/translate.c:801
801     ctx->base.pc_next = ctx->pc_succ_insn;
(gdb) n
803     if (ctx->base.is_jump == DISAS_NEXT) {
(gdb)
```

为什么使用n不使用si

si是执行当前指令，然后停在下一条指令上，可以精确到汇编指令级别，而**n(next)**是执行到下一行源代码，然后停下来。如果当前行有函数调用，会直接执行整个函数，不进入函数内部，可以快速跟踪程序逻辑。

整合后代码如下所示

trans_ecall

```
1  trans_ecall (
2  ctx=0x77fc71280800, a=0x77fc71280700)
3  {
4      generate_exception(ctx, RISCV_EXCP_U_ECALL);
5      exit_tb(ctx); /* no chaining */
6      ctx->base.is_jump = DISAS_NORETURN;
7      return true;
```

这个函数的作用**翻译期生成 `ecall` 异常处理的 TCG 代码**，它根据当前指令上下文 `ctx` 调用 `generate_exception(ctx, RISC_V_EXCP_U_ECALL)` 来生成触发用户态 `ecall` 的异常动作，并通过 `exit_tb(ctx)` 和 `ctx->base.is_jump = DISAS_NORETURN` 表示 TB (Translation Block) 在执行到这里后不会继续顺序执行下一条指令，从而完成 `ecall` 的翻译。

可以看到紧接着还调用了一个函数 `decode_insn32`, `decode_opc`, `riscv_tr_translate_insn`，作用分别为：

`decode_insn32(ctx, insn)`：负责解码一条 32 位 RISC-V 指令，并调用对应的翻译函数（比如 `trans_ecall`）生成 TCG 代码，如果指令已经处理（如异常或系统调用）就返回 `true`，否则继续正常翻译。

`decode_opc(ctx)`：作为指令解码的顶层调度器，根据当前指令 `ctx->opcode` 类型选择调用合适的解码函数（如 `decode_insn32`），完成整条指令的翻译派发。

`riscv_tr_translate_insn(dcbase, cpu)`：作为单条指令翻译的入口函数，初始化翻译上下文、设置下一条指令地址，然后调用 `decode_opc()` 执行指令翻译，最终生成可在 host 上执行的 TCG 代码块，为 QEMU 执行做准备。

2、riscv_raise_exception

接下来我们再输入 `c` 继续运行，发现在 `riscv_raise_exception` 出现了中断，结果如下

```
Thread 3 "qemu-system-ris" hit Breakpoint 2, riscv_raise_exception
(env=0x8000000, exception=0, pc=0)
at /home/osaq/qemu-4.1.1-debug/target/riscv/op_helper.c:30
30 {
(gdb) n
31     CPUState *cs = env_cpu(env);
(gdb) n
32     qemu_log_mask(CPU_LOG_INT, "%s: %d\n", __func__, except
ion);
(gdb) n
33     cs->exception_index = exception;
(gdb) n
34     cpu_loop_exit_restore(cs, pc);
(gdb) n
0x00005c9908d0ada4 in cpu_exec (cpu=0x5c993b4d3590)
at /home/osaq/qemu-4.1.1-debug/accel/tcg/cpu-exec.c:689
689     if (sigsetjmp(cpu->jmp_env, 0) != 0) {
(gdb) n
699         g_assert(cpu == current_cpu);
(gdb) n
700         g_assert(cc == CPU_GET_CLASS(cpu));
(gdb) n
705         if (qemu_mutex_iothread_locked()) {
(gdb) n
708             assert_no_pages_locked();
(gdb) n
712         while (!cpu_handle_exception(cpu, &ret)) {
(gdb) n

Thread 3 "qemu-system-ris" hit Breakpoint 4, riscv_cpu_do_interrupt
(cs=0x5c990989eec0 <qemu_global_mutex>)
```

```

1 riscv_raise_exception (env=0x800000, exception=0, pc=0)
2 {
3     CPUState *cs = env_cpu(env);
4     qemu_log_mask(CPU_LOG_INT, "%s: %d\n", __func__, exception);
5     cs->exception_index = exception;
6     cpu_loop_exit_restore(cs, pc);
7     cpu_exec (cpu=0x5c993b4d3590)
8     if (sigsetjmp(cpu->jmp_env, 0) != 0) {
9         g_assert(cpu == current_cpu);
10        g_assert(cc == CPU_GET_CLASS(cpu));
11        if (qemu_mutex_iothread_locked()) {
12            assert_no_pages_locked();
13        while (!cpu_handle_exception(cpu, &ret)) {

```

`riscv_raise_exception` 的核心作用是“触发并上报异常并打断执行流”，而不是处理异常本身。当执行期遇到 `ecall` 等同步异常时，`riscv_raise_exception` 并不直接处理异常，而是先把异常号写入 `CPUState` 的 `exception_index`，然后调用 `cpu_loop_exit_restore` 通过 `longjmp` 强制退出当前正在执行的 TB，跳回 `cpu_exec` 中事先用 `sigsetjmp` 建立的恢复点；此时 `sigsetjmp` 返回非 0，`cpu_exec` 进入异常路径，在 `while (!cpu_handle_exception(...))` 循环中根据刚设置的 `exception_index` 真正完成异常分发、特权级切换和跳转到异常向量。

3、riscv_cpu_do_interrupt

然后qemu代码又停在了riscv_cpu_do_interrupt处

```

Thread 3 "qemu-system-ris" hit Breakpoint 4, riscv_cpu_do_interrupt
(cs=0x5c990989eec0 <qemu_global_mutex>)
  at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:504
504 {
(gdb) n
507     RISCVCPU *cpu = RISCV_CPU(cs);
(gdb) n
508     CPURISCVState *env = &cpu->env;
(gdb) n
513     bool async = !(cs->exception_index & RISCV_EXCP_INT_FL
AG);
(gdb) n
514     target_ulong cause = cs->exception_index & RISCV_EXCP_I
NT_MASK;
(gdb) n
515     target_ulong deleg = async ? env->mideleg : env->medele
g;
(gdb) n
516     target_ulong tval = 0;
(gdb) n
525     if (!async) {
(gdb) n
540         break;
(gdb) n
543         if (cause == RISCV_EXCP_U_ECALL) {
(gdb) n
544             assert(env->priv <= 3);
(gdb) n
545             cause = ecall_cause_map[env->priv];
(gdb) n
549             trace_riscv_trap(env->mhartid, async, cause, env->pc, t
val, cause < 16 ?
(gdb) n
550             (async ? riscv_intr_names : riscv_excp_names)[cause
] : "(unknown)");

```

```

(gdb) n
549      trace_riscv_trap(env->mhartid, async, cause, env->pc, t
val, cause < 16 ?
(gdb) n
552      if (env->priv <= PRV_S &&
(gdb) n
553          cause < TARGET_LONG_BITS && ((deleg >> cause) &
1)) {
(gdb) n
569      target_ulong s = env->mstatus;
(gdb) n
570      s = set_field(s, MSTATUS_MPIE, env->priv_ver >= PRI
V_VERSION_1_10_0 ?
(gdb) n
572      s = set_field(s, MSTATUS_MPP, env->priv);
(gdb) n
573      s = set_field(s, MSTATUS_MIE, 0);
(gdb) n
574      env->mstatus = s;
(gdb) n
575      env->mcause = cause | ~(((target_ulong)-1) >> async
);
(gdb) n
576      env->mepc = env->pc;
(gdb) n
577      env->mbadaddr = tval;
(gdb) n
578      env->pc = (env->mtvec >> 2 << 2) +
(gdb) n
579          ((async && (env->mtvec & 3) == 1) ? cause * 4 :
0);
(gdb) n
578      env->pc = (env->mtvec >> 2 << 2) +
(gdb) n
580      riscv_cpu_set_mode(env, PRV_M);
(gdb) n

```

riscv_cpu_do_interrupt

```

1  riscv_cpu_do_interrupt(cs=0x5c990989eec0 <qemu_global_mutex>)
2  {
3      RISCVCPU *cpu = RISCVCPU(cs);
4      CPURISCVState *env = &cpu->env;
5      bool async = !(cs->exception_index & RISCV_EXCP_INT_FLAG);
6      target_ulong cause = cs->exception_index & RISCV_EXCP_INT_MASK;
7      target_ulong deleg = async ? env->mideleg : env->medeleg;
8      target_ulong tval = 0;
9      if (!async) {
10         break;
11         if (cause == RISCV_EXCP_U_ECALL) {
12             assert(env->priv <= 3);
13             cause = ecall_cause_map[env->priv];
14             trace_riscv_trap(env->mhartid, async, cause, env->pc, tval, cause
< 16 ?
15                 (async ? riscv_intr_names : riscv_excp_names)[cause] : "
(unknown)");
16             trace_riscv_trap(env->mhartid, async, cause, env->pc, tval, cause
< 16 ?

```

```

17         if (env->priv <= PRV_S &&
18             cause < TARGET_LONG_BITS && ((deleg >> cause) & 1)) {
19             target_ulong s = env->mstatus;
20             s = set_field(s, MSTATUS_MPIE, env->priv_ver >=
PRIV_VERSION_1_10_0 ?
21                 s = set_field(s, MSTATUS_MPP, env->priv);
22                 s = set_field(s, MSTATUS_MIE, 0);
23                 env->mstatus = s;
24                 env->mcause = cause | ~(((target_ulong)-1) >> async);
25                 env->mepc = env->pc;
26                 env->mbadaddr = tval;
27                 env->pc = (env->mtvec >> 2 << 2) + ((async && (env->mtvec & 3)) ==
1) ? cause * 4 : 0);
28                 env->pc = (env->mtvec >> 2 << 2) +
29                 riscv_cpu_set_mode(env, PRV_M);
30     }

```

riscv_cpu_do_interrupt的主要作用是依据异常类型与委派规则，完成异常原因确定、上下文保存、入口地址计算和特权级切换。它从 `CPUState` 里取出 `exception_index`，先判断这是异步中断还是同步异常（如 `ecall`），并提取异常号 `cause`，对 `U/S/M ecall` 这类同步异常按当前特权级重映射成架构规定的异常原因；随后根据 `medeleg/mideleg` 判断该异常是否应当**下放（delegate）**给更低特权级处理，如果不下放或需在 M 模式处理，就保存当前特权级和中断使能到 `mstatus`（更新 `MPIE/MPP/MIE`），填写 `mcause`、`mepc`、`mbadaddr(tval)` 等异常上下文寄存器，并根据 `mtvec` 的模式（direct/vectored）计算异常入口地址更新 `pc`，最后调用 `riscv_cpu_set_mode(env, PRV_M)` 切换到目标特权级开始执行异常处理代码。

4、riscv_cpu_set_mode

接下来调用riscv_cpu_set_mode函数

```

Thread 3 "qemu-system-ris" hit Breakpoint 5, riscv_cpu_set_mode (
    env=0x0, newpriv=1898449184)
    at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:126
126     {
(gdb) n
127         if (newpriv > PRV_M) {
(gdb) n
130         if (newpriv == PRV_H) {
(gdb) n
134         env->priv = newpriv;
(gdb) n
144         env->load_res = -1;
(gdb) n
145     }
(gdb) n
riscv_cpu_do_interrupt (cs=0x5c993b4d3590)
    at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:590
590         cs->exception_index = EXCP_NONE; /* mark handled to qem
u */
(gdb) n
591     }

```



```

riscv_cpu_set_mode

1  riscv_cpu_set_mode (env=0x0, newpriv=1898449184)
2      {
3          if (newpriv > PRV_M) {
4              if (newpriv == PRV_H) {
5                  env->priv = newpriv;
6                  env->load_res = -1;
7              }
8          riscv_cpu_do_interrupt (cs=0x5c993b4d3590)
9              cs->exception_index = EXCP_NONE; /* mark handled to qemu */
10         }

```

riscv_cpu_set_mode的主要作用是异常处理完成后将 CPU 切换到目标特权级并清除异常标记，表示异常已被 QEMU 正确处理，随后从新的特权级和 PC 继续执行。

在异常处理过程中，riscv_cpu_do_interrupt 在完成异常原因判定、异常上下文保存以及异常入口地址计算后，通过调用 riscv_cpu_set_mode 将 CPU 的当前特权级切换为目标特权级（如进入 M 模式或 S 模式），riscv_cpu_set_mode 内部首先检查新特权级是否合法，然后更新 env->priv 表示当前运行模式，并清空 load_res 以失效可能残留的原子访存状态；当异常跳转准备完成后，riscv_cpu_do_interrupt 将 cs->exception_index 置为 EXCP_NONE，告知 QEMU 该异常已经被正确处理，后续执行将从新的 PC 和特权级继续进行，而不会再次重复处理该异常。

5、cpu_handle_interrupt

最后调用了cpu_handle_interrupt函数

```

cpu_handle_exception (cpu=0x5c993b4d3590, ret=0x77fc712809ac)
    at /home/osaq/qemu-4.1.1-debug/accel/tcg/cpu-exec.c:507
507         qemu_mutex_unlock_iothread();
(gdb) n
508         cpu->exception_index = -1;
(gdb) n
517         return false;
(gdb) n
518     }
(gdb) n

```

```

cpu_handle_interrupt

1  cpu_handle_exception (cpu=0x5c993b4d3590, ret=0x77fc712809ac)
2      qemu_mutex_unlock_iothread();
3      cpu->exception_index = -1;
4      return false;
5  }

```

cpu_handle_interrupt的主要作用是在异常已经被体系结构相关代码处理完之后，释放全局 I/O 线程锁并清除 CPU 的异常状态，通知 `cpu_exec` 可以结束异常处理循环并继续正常执行。它会先调用 `qemu_mutex_unlock_iothread()` 释放在异常处理期间持有的 I/O 线程锁，确保系统其余设备和线程可以继续运行；随后将 `cpu->exception_index` 置为 `-1`，表示当前 CPU 已不存在待处理的异常；最后返回 `false`，使 `cpu_exec` 中的 `while (!cpu_handle_exception(...))` 循环退出，从而让 QEMU 从异常处理路径回到正常的 TB 执行流程。

最后我们在尝试输入n单步调试，看看后面会发生上面，发现这里再次回到了qemu的主循环 `cpu_exec`,

```
cpu_exec (cpu=0x5c993b4d3590)
  at /home/osaq/qemu-4.1.1-debug/accel/tcg/cpu-exec.c:712
712     while (!cpu_handle_exception(cpu, &ret)) {
(gdb) n
713         TranslationBlock *last_tb = NULL;
(gdb) n
714         int tb_exit = 0;
(gdb) n
716         while (!cpu_handle_interrupt(cpu, &last_tb)) {
(gdb) n

Thread 3 "qemu-system-ris" hit Breakpoint 3, cpu_handle_interrupt
(cpu=0x5c993b4d3590, last_tb=0x77fc712809b8)
  at /home/osaq/qemu-4.1.1-debug/accel/tcg/cpu-exec.c:523
523     CPUClass *cc = CPU_GET_CLASS(cpu);
(gdb) n
530     atomic_mb_set(&cpu_neg(cpu)->icount_decr.u16.high, 0);
(gdb) n
532     if (unlikely(atomic_read(&cpu->interrupt_request))) {
(gdb) n
534         qemu_mutex_lock_iothread();
(gdb) n
535         interrupt_request = cpu->interrupt_request;
(gdb) n
536         if (unlikely(cpu->singlestep_enabled & SSTEP_NOIRQ)
) {
(gdb) n
540             if (interrupt_request & CPU_INTERRUPT_DEBUG) {
(gdb) n
546             if (replay_mode == REPLAY_MODE_PLAY && !replay_has_
interrupt()) {
(gdb) n
548             } else if (interrupt_request & CPU_INTERRUPT_HALT)
{
(gdb) n
568             else if (interrupt_request & CPU_INTERRUPT_RESET) {
```

当我们执行到 `trans_eca11` 并触发异常后：`cpu_handle_exception` 调用 → 内核异常入口被设置，然后循环回到这里继续执行 TB，但现在 TB 会从 **内核异常入口** 开始。

接着一直在重复这样的循环(具体这部分会在后面进行更详细的分析)

2.4 设置sret相关断点

接下来我们先删除前面设置的断点（防止后面运行中遇到不必要的中断），然后回到终端3（调试内核的gdb),找到源文件中sret指令的位置，并直接对源文件打下断点，

b kern/trap/trapentry.S: __trapret

如下图所示

```
(gdb) b kern/trap/trapentry.S:__trapret
Breakpoint 3 at 0xffffffffc0200ec8: file kern/trap/trapentry.S, line 131.
(gdb)
```

然后，我们输入c运行，代码成功停在了我们设置的断点处

```
Breakpoint 3, __trapret () at kern/trap/trapentry.S:131
131      RESTORE_ALL
(gdb) x/7i $pc
=> 0xffffffffc0200ec8 <__trapret>:      ld      s1,256(sp)
0xffffffffc0200eca <__trapret+2>:      ld      s2,264(sp)
0xffffffffc0200ecc <__trapret+4>:      andi     s0,s1,256
0xffffffffc0200ed0 <__trapret+8>:      bnez     s0,0xffffffffc0200ed8 <__trapret+16>
0xffffffffc0200ed2 <__trapret+10>:     addi     s0,sp,288
0xffffffffc0200ed4 <__trapret+12>:     csrw     sscratch,s0
0xffffffffc0200ed8 <__trapret+16>:     csrw     sstatus,s1
```

我们还是使用命令x/7i \$pc查看从当前PC位置开始的7条指令反汇编指令，我们发现这与源代码中的指令似乎不一致

sret源码

```
1  __trapret:
2      RESTORE_ALL
3      # return from supervisor call
4      sret
```

我们发现这里没有出现sret指令，而前面syscall中却出现了ecall指令。

因为在 uCore GDB 中，`ecall` 能看到是因为它是用户态触发系统调用的指令，发生在用户态汇编指令流中，你用 GDB 单步时可以停在这条指令上；执行 `ecall` 后，CPU 进入异常处理，QEMU 调用 `generate_exception(ctx, RISC_V_EXCP_U_ECALL)` 抛出异常，所以你能在 GDB 中看到 `trans_ecall` 或 `riscv_raise_exception`。

而 `sret` 是内核态返回用户态的指令，它在 trap 处理程序里执行，QEMU 使用 TCG 将 RISC-V 指令翻译成宿主机码，并通过函数 `riscv_cpu_set_mode(env, PRV_U)` 完成特权级切换；因此在用户态 GDB 视角中，你只会看到内核的 `__trapret` 汇编或宿主指令流，而不会看到真实的 sret 指令执行。

2.5 为qemu设置sret相关断点

接着我们还是再回到终端2，在大模型的帮助下，我们知道了qemu处理sret主要的函数为

`trans_sret`、`helper_sret` 和 `riscv_cpu_set_mode`，大致流程如下

1、翻译期生成代码：`trans_sret`

在 TCG 翻译阶段，将 guest 的 `sret` 指令翻译成 host 可执行的 TCG 代码；生成对 `helper_sret` 的调用，并设置 TB 不会顺序执行下一条指令（`DISAS_NORETURN`）。

2、执行期完成特权恢复： `helper_sret`

在 host 执行期间，`helper_sret` 负责真正执行 `sret` 的语义。它读取 `sepc` 作为返回地址，更新 `mstatus` 中的 `SPP/SPIE` 等状态位，并准备恢复上一个特权级；调用 `riscv_cpu_set_mode` 切换到上一次保存的特权级。

3、特权级切换： `riscv_cpu_set_mode`

更新 CPU 当前特权级，清除原子访存状态，确保后续执行在正确特权级下运行。

于是我们在这三个位置上打下断点，输入

b trans_sret

b helper_sret

b riscv_cpu_set_mode

然后输入c继续运行

```
(gdb) b trans_sret
Breakpoint 1 at 0x5ab1996fc22d: file /home/osaq/qemu-4.1.1-debug/target/riscv/insn_trans/trans_privileged.inc.c, line 44.
(gdb) b helper_sret
Breakpoint 2 at 0x5ab1996feb63: file /home/osaq/qemu-4.1.1-debug/target/riscv/op_helper.c, line 75.
(gdb) b riscv_cpu_set_mode
Breakpoint 3 at 0x5ab1996ff683: file /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c, line 126.
(gdb) c
Continuing.
```

2.6 qemu处理sret调试过程

接着我们回到终端3，输入c继续运行，结果如下

```
(gdb) c
Continuing.

Breakpoint 3, __trapret () at kern/trap/trapentry.S:131
131      RESTORE_ALL
(gdb) c
Continuing.

Continuing.
[Switching to Thread 0x727660901700 (LWP 46524)]
Thread 3 "qemu-system-ris" hit Breakpoint 5, riscv_cpu_set_mode (
  env=0x5cb41da70fa0, newpriv=4)
  at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:126
126      {
(gdb)
```

可以看到，终端2（调试qemu的gdb）成功捕获了一个riscv_cpu_set_mode断点

为什么这里最先捕获的是riscv_cpu_set_mode断点呢，按前面的分析不应该是先进入trans_sret函数吗？

大模型的回答(整合版)

因为 `trans_sret` 发生在 TCG 的翻译期，而为我们捕获到的 `riscv_cpu_set_mode` 属于执行期的特权级切换：当 QEMU 开始执行包含 `sret` 的 TB 时，CPU 可能仍处在异常返回路径或前一

次异常处理中，执行期会先根据保存的异常上下文调用 `riscv_cpu_set_mode` 恢复特权级；而 `trans_sret` 只是翻译阶段生成代码，只有在重新翻译 TB、且执行流真正走到 `sret` 指令时才会命中断点，所以调试时看起来总是先命中 `riscv_cpu_set_mode`，与前面的流程分析是不违背的。

接着我们再输入多个c，找到了一个一个处理sret的基本流程，如下图所示

```
Thread 3 "qemu-system-ris" hit Breakpoint 1, trans_sret (
    ctx=0x79c8eca81800, a=0x79c8eca81700)
    at /home/osaq/qemu-4.1.1-debug/target/riscv/insn_trans/trans_privileged.inc.c:44
44      {
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 2, helper_sret (
    env=0x141, cpu_pc_deb=105557411233792)
    at /home/osaq/qemu-4.1.1-debug/target/riscv/op_helper.c:75
75      {
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 3, riscv_cpu_set_mode (
    env=0x5ab1da0e1fa0, newpriv=4)
    at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:126
126     {
(gdb) c
Continuing.
```

qemu确实先后调用了`trans_sret`、`helper_sret`、`riscv_cpu_set_mode`这三个函数，接下来我们进行单步调试，分析每个函数的内部逻辑

1、trans_sret

```

Thread 3 "qemu-system-ris" hit Breakpoint 6, trans_sret (
    ctx=0x77fc71280800, a=0x77fc71280700)
    at /home/osaq/qemu-4.1.1-debug/target/riscv/insn_trans/trans_privileged.inc.c:44
44      {
(gdb) n
46          tcg_gen_movi_tl(cpu_pc, ctx->base.pc_next);
(gdb) n
48          if (has_ext(ctx, RVS)) {
(gdb) n
49              gen_helper_sret(cpu_pc, cpu_env, cpu_pc);
(gdb) n
50              exit_tb(ctx); /* no chaining */
(gdb) n
51              ctx->base.is_jump = DISAS_NORETURN;
(gdb) n
55              return true;
(gdb) n
59          }
(gdb) n
decode_insn32 (ctx=0x77fc71280800, insn=270532723)
    at target/riscv/decode_insn32.inc.c:1638
1638          if (trans_sret(ctx, &u.f_empty)) return true;
(gdb) n
1731      }
(gdb) n
decode_opc (ctx=0x77fc71280800)
    at /home/osaq/qemu-4.1.1-debug/target/riscv/translate.c:746
746          if (!decode_insn32(ctx, ctx->opcode)) {
(gdb) n
750      }
(gdb) n
riscv_tr_translate_insn (dcbase=0x77fc71280800,
    cpu=0x5c993b4d3590)
    at /home/osaq/qemu-4.1.1-debug/target/riscv/translate.c:801
801          ctx->base.pc_next = ctx->pc_succ_insn;

```

trans_sret

```

1  trans_sret (ctx=0x77fc71280800, a=0x77fc71280700)
2      {
3          tcg_gen_movi_tl(cpu_pc, ctx->base.pc_next);
4          if (has_ext(ctx, RVS)) {
5              gen_helper_sret(cpu_pc, cpu_env, cpu_pc);
6              exit_tb(ctx); /* no chaining */
7              ctx->base.is_jump = DISAS_NORETURN;
8              return true;
9          }

```

trans_sret的主要作用是在翻译期将 `sret` 指令生成对应的 TCG 代码，设置异常返回的处理逻辑并标记 TB 不会顺序执行下一条指令，从而实现从特权态返回到上一个特权级的控制流恢复。

它先用 `tcg_gen_movi_tl` 将下一条指令地址写入 `cpu_pc`，如果 CPU 支持 RVS 扩展，则调用 `gen_helper_sret` 生成执行期 helper 代码来执行实际的特权级切换和状态恢复；随后调用 `exit_tb(ctx)` 表示当前 TB 执行到这里后不再顺序执行，设置 `ctx->base.is_jump = DISAS_NORETURN` 并返回 `true`，通知翻译器该指令已处理完毕并完成异常返回的翻译。

2、helper_sret

```
Thread 3 "qemu-system-ris" hit Breakpoint 7, helper_sret (
  env=0x141, cpu_pc_deb=105559558718790)
  at /home/osaq/qemu-4.1.1-debug/target/riscv/op_helper.c:75
75     {
(gdb) n
76     if (!(env->priv >= PRV_S)) {
(gdb) n
80     target_ulong retpc = env->sepc;
(gdb) n
81     if (!riscv_has_ext(env, RVC) && (retpc & 0x3)) {
(gdb) n
85     if (env->priv_ver >= PRIV_VERSION_1_10_0 &&
(gdb) n
86         get_field(env->mstatus, MSTATUS_TSR)) {
(gdb) n
85     if (env->priv_ver >= PRIV_VERSION_1_10_0 &&
(gdb) n
90     target_ulong mstatus = env->mstatus;
(gdb) n
91     target_ulong prev_priv = get_field(mstatus, MSTATUS_SPP
);
(gdb) n
92     mstatus = set_field(mstatus,
(gdb) n
96     mstatus = set_field(mstatus, MSTATUS_SPIE, 0);
(gdb) n
97     mstatus = set_field(mstatus, MSTATUS_SPP, PRV_U);
(gdb) n
98     riscv_cpu_set_mode(env, prev_priv);
(gdb) n

Thread 3 "qemu-system-ris" hit Breakpoint 8, riscv_cpu_set_mode (
  env=0x5c993b4dbfa0, newpriv=4)
  at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:126
```

代码块

```
1  helper_sret (env=0x141, cpu_pc_deb=105559558718790)
2      {
3          if (!(env->priv >= PRV_S)) {
4              target_ulong retpc = env->sepc;
5              if (!riscv_has_ext(env, RVC) && (retpc & 0x3)) {
6                  if (env->priv_ver >= PRIV_VERSION_1_10_0 &&
7                      get_field(env->mstatus, MSTATUS_TSR)) {
8                      if (env->priv_ver >= PRIV_VERSION_1_10_0 &&
9                          target_ulong mstatus = env->mstatus;
10                     target_ulong prev_priv = get_field(mstatus, MSTATUS_SPP);
11                     mstatus = set_field(mstatus,
12                         mstatus = set_field(mstatus, MSTATUS_SPIE, 0);
13                         mstatus = set_field(mstatus, MSTATUS_SPP, PRV_U);
14                         riscv_cpu_set_mode(env, prev_priv);
15 riscv_cpu_set_mode (env=0x5c993b4dbfa0, newpriv=4)
```


`helper_sret`函数的主要作用为在执行期完成 `sret` 指令的实际特权级恢复和状态寄存器更新，将CPU 从当前特权级返回到上一次保存的特权级，并设置下一条执行指令的地址。

`helper_sret` 首先检查当前特权级是否合法，然后读取 `sepc` 作为返回地址，处理压缩指令对齐和特权版本相关的约束；接着根据 `mstatus` 保存的 `SPP`（上一次特权级）恢复特权级，同时更新 `SPIE/SIE/SPP` 等状态位以正确恢复中断使能和状态；最后调用

`riscv_cpu_set_mode(env, prev_priv)` 切换到上一次特权级，从而完成 `sret` 指令的控制流和特权状态恢复。

3、riscv_cpu_set_mode

然后接下来是`riscv_cpu_set_mode`函数，它主要用于用于 **切换 CPU 当前的特权级并更新相关状态寄存器**，我们在前面的`ecall`处理过程中已经详细分析过了这个函数，这里就不再赘述了，就简单看一下它的处理过程。

```
Thread 3 "qemu-system-ris" hit Breakpoint 8, riscv_cpu_set_mode (
  env=0x5c993b4dbfa0, newpriv=4)
  at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:126
126      {
(gdb) n
127          if (newpriv > PRV_M) {
(gdb) n
130          if (newpriv == PRV_H) {
(gdb) n
134          env->priv = newpriv;
(gdb) n
144          env->load_res = -1;
(gdb) n
145      }
(gdb) n
helper_sret (env=0x5c993b4dbfa0, cpu_pc_deb=18446744072637910814)
  at /home/osaq/qemu-4.1.1-debug/target/riscv/op_helper.c:99
99      env->mstatus = mstatus;
(gdb) n
101      return retpc;
(gdb) n
102  }
(gdb) n
0x000077fc6a000d8f in code_gen_buffer ()
(gdb) n
Single stepping until exit from function code_gen_buffer,
which has no line number information.
0x000077fc6a000016 in ?? ()
(gdb) n
Cannot find bounds of current function
(gdb) n
Cannot find bounds of current function
```

这里我们还发现`helper_sret`函数结束后，后面紧接着还调用了`code_gen_buffer`并且再次输入`n`出现了一个奇怪的输出

代码块

```
1  (gdb) n
2  0x0000727660902d8f in code_gen_buffer ()
3  (gdb) n
4  Single stepping until exit from function code_gen_buffer,
```

```
5  which has no line number information.
6  0x0000727660902016 in ?? ()
7  Cannot find bounds of current function
```

在**大模型**的帮助下我们知道：因为在 QEMU 的 TCG 执行模型中，`helper_sret` 只是生成或执行了 **特权级切换和状态恢复的 helper 代码**，而 `code_gen_buffer` 是 QEMU TCG 执行环境的底层执行循环，用于调度 TB 和 host 指令，所以在执行 `sret` 后回到 `code_gen_buffer` 继续管理 TB 流程。

而我们这个奇怪的输出是因为我们正在单步执行 **TCG 动态生成的 host 机器码**，这些代码没有源文件信息，也没有明确的函数边界，GDB 无法显示行号，只能显示地址 `??`。这是 QEMU 调试中非常常见的现象，并不表示程序异常。

最后我们尝试再运行c观察终端3是否会再次在一个sret处停下，但结果发现，终端3并没有变化，我们在终端2输入了好多次c发现在一些相同的函数处中断了很多次，

```
Thread 3 "qemu-system-ris" hit Breakpoint 5, riscv_cpu_set_mode (
    env=0x0, newpriv=1620052256)
    at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:126
126    {
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 5, riscv_cpu_set_mode (
    env=0x5cb41da70fa0, newpriv=4)
    at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:126
126    {
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 6, riscv_cpu_do_interrupt
(cs=0x727660900910)
    at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:504
504    {
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 5, riscv_cpu_set_mode (
    env=0x0, newpriv=4601364328)
    at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:126
126    {
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 5, riscv_cpu_set_mode (
    env=0x5cb41da70fa0, newpriv=4)
    at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:126
126    {
(gdb) c
Continuing.
```

即表明，在qemu在对一次sret的处理过程中，多次调用了`riscv_cpu_set_mode`和`riscv_cpu_do_interrupt`这两个函数，我们**询问大模型**得知：因为 `sret` 涉及 **从当前特权级返回上一次特权级**，整个过程可能包括：先恢复中间状态寄存器（`mstatus/SPP/SPIE`）、切换特权级、处理可能的挂起中断，每个状态更新或特权级切换都会调用 `riscv_cpu_set_mode`，而每次需要根据当前异常/中断上下文处理跳转或 pending 异常时都会调用 `riscv_cpu_do_interrupt`，所以看起来像多次调用，实际上是 QEMU 在不同阶段分别完成 **特权级恢复、异常分发和中断处理** 的正常流程。

后来我们再一直尝试输入c，发现接下来一直在调用cpu_exec这两个函数

```
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 8, riscv_cpu_set_mode (
    env=0x0, newpriv=4448382824)
    at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:126
126      {
(gdb) c
Continuing.
0x00005c9908d0ada4 in cpu_exec (cpu=0x5c993b4d3590)
    at /home/osaq/qemu-4.1.1-debug/accel/tcg/cpu-exec.c:689
689      if (sigsetjmp(cpu->jmp_env, 0) != 0) {
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 8, riscv_cpu_set_mode (
    env=0x0, newpriv=1898449184)
    at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:126
126      {
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 8, riscv_cpu_set_mode (
    env=0x5c993b4dbfa0, newpriv=4)
    at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:126
126      {
(gdb) c
Continuing.
0x00005c9908d0ada4 in cpu_exec (cpu=0x5c993b4d3590)
    at /home/osaq/qemu-4.1.1-debug/accel/tcg/cpu-exec.c:689
```

接着我们单步执行，结果如下

```

0x00005c9908d0ada4 in cpu_exec (cpu=0x5c993b4d3590)
  at /home/osaq/qemu-4.1.1-debug/accel/tcg/cpu-exec.c:689
689         if (sigsetjmp(cpu->jmp_env, 0) != 0) {
(gdb) n
699             g_assert(cpu == current_cpu);
(gdb) n
700             g_assert(cc == CPU_GET_CLASS(cpu));
(gdb) n
705             if (qemu_mutex_iothread_locked()) {
(gdb) n
708                 assert_no_pages_locked();
(gdb) n
712             while (!cpu_handle_exception(cpu, &ret)) {
(gdb) n

Thread 3 "qemu-system-ris" hit Breakpoint 8, riscv_cpu_set_mode (
  env=0x0, newpriv=1898449184)
  at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:126
126         {
(gdb) n
127             if (newpriv > PRV_M) {
(gdb) n
130             if (newpriv == PRV_H) {
(gdb) n
134                 env->priv = newpriv;
(gdb) n
144                 env->load_res = -1;
(gdb) n
145             }
(gdb) n
riscv_cpu_do_interrupt (cs=0x5c993b4d3590)
  at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:590
590         cs->exception_index = EXCP_NONE; /* mark handled to qem
u */

```

代码块

```

1  cpu_exec (cpu=0x5c993b4d3590)
2      if (sigsetjmp(cpu->jmp_env, 0) != 0) {
3          g_assert(cpu == current_cpu);
4          g_assert(cc == CPU_GET_CLASS(cpu));
5          if (qemu_mutex_iothread_locked()) {
6              assert_no_pages_locked();
7              while (!cpu_handle_exception(cpu, &ret)) {
8  riscv_cpu_set_mode (env=0x0, newpriv=1898449184)
9      {
10         if (newpriv > PRV_M) {
11         if (newpriv == PRV_H) {
12             env->priv = newpriv;
13             env->load_res = -1;
14         }
15  riscv_cpu_do_interrupt (cs=0x5c993b4d3590)
16         cs->exception_index = EXCP_NONE; /* mark handled to qemu */
17  cpu_handle_exception (cpu=0x5c993b4d3590, ret=0x77fc712809ac)
18         qemu_mutex_unlock_iothread();
19         cpu->exception_index = -1;
20         return false;
21     }

```

这其实是QEMU 在执行 guest 指令时 **异常处理的完整循环**：当 `cpu_exec` 检测到通过 `siglongjmp` 跳回的异常（`sigsetjmp` 返回非 0）时，会先进行一系列一致性检查和锁验证，然后进入 `while (!cpu_handle_exception(...))` 循环，在循环中调用 `riscv_cpu_do_interrupt` 根据 `exception_index` 完成异常分发、特权级切换和上下文保存（期间可能调用 `riscv_cpu_set_mode` 切换特权级）；当异常处理完成后，`cpu_handle_exception` 会释放 I/O 线程锁、清空异常标记并返回 `false`，使循环结束，控制流回到正常 TB 执行，从而实现 QEMU 对同步异常或中断的正确捕获、处理和恢复。

接下来还是再进行单步执行，最后得到了以下这段执行逻辑

```
cpu_exec (cpu=0x5c993b4d3590)
  at /home/osaq/qemu-4.1.1-debug/accel/tcg/cpu-exec.c:712
712     while (!cpu_handle_exception(cpu, &ret)) {
(gdb) n
713         TranslationBlock *last_tb = NULL;
(gdb) n
714         int tb_exit = 0;
(gdb) n
716         while (!cpu_handle_interrupt(cpu, &last_tb)) {
(gdb) n
717             uint32_t cflags = cpu->cflags_next_tb;
(gdb) n
725             if (cflags == -1) {
(gdb) n
726                 cflags = curr_cflags();
(gdb) n
731             tb = tb_find(cpu, last_tb, tb_exit, cflags);
(gdb) n
732             cpu_loop_exec_tb(cpu, tb, &last_tb, &tb_exit);
(gdb) n
735             align_clocks(&sc, cpu);
(gdb) n
716             while (!cpu_handle_interrupt(cpu, &last_tb)) {
(gdb) n
717                 uint32_t cflags = cpu->cflags_next_tb;|
(gdb) n
725                 if (cflags == -1) {
```

最后我们发现 `cpu_exec` 函数中一直在执行一个重复的while循环代码，仔细查看这段循环代码

```
cpu_exec
1  while (!cpu_handle_interrupt(cpu, &last_tb)) {
2      uint32_t cflags = cpu->cflags_next_tb;
3      if (cflags == -1) {
4          cflags = curr_cflags();
5      }
6      tb = tb_find(cpu, last_tb, tb_exit, cflags);
7      cpu_loop_exec_tb(cpu, tb, &last_tb, &tb_exit);
8      align_clocks(&sc, cpu);
9  }
```

其实这正是QEMU CPU的主循环（`cpu_exec`）在执行**指令翻译和执行**，它是qemu在模拟整个**Guest CPU**的运行。

具体分析如下

1、`cpu_handle_interrupt` 判断是否有需要处理的中断/异常

`cpu_handle_interrupt(cpu, &last_tb)` 返回 `true` 表示有异常/中断需要处理 → 跳出循环去处理它；返回 `false` 表示没有异常/中断 → 继续执行 Guest 指令。

所以当没有新的中断或异常时，这个循环会不断执行。

2、TB（Translation Block）查找和执行

TB

```
1  tb = tb_find(cpu, last_tb, tb_exit, cflags);
2  cpu_loop_exec_tb(cpu, tb, &last_tb, &tb_exit);
```

`tb_find`：查找/生成指令块（Translation Block, TB），这块 TB 对应一段连续的 Guest 指令。

`cpu_loop_exec_tb`：执行这个 TB，实际上就是执行 Guest 指令。

每执行完一个 TB，循环会继续，直到遇到新的中断或异常。

整体来看，这段一直重复的代码就是**QEMU TCG**的核心模拟循环，即**执行一个 TB → 更新 PC → 下一次循环 → 执行下一个 TB**，如果Guest CPU 没有触发新的异常/中断，循环永远不会跳出。

三、指令翻译（TCG Translation）

要求：在执行`ecall`和`sret`这类汇编指令的时候，qemu进行了很关键的一步——指令翻译（TCG Translation），了解一下这个功能，思考一下另一个双重gdb调试的实验是否也涉及到了一些相关的内容。

TCG 的目标是把 guest 的基本块（translation block, TB）编译为高效的 host 代码，内联常用操作（包括内存访问的快速 TLB 检查）从而显著减少函数调用和分支开销。

指令翻译正是由刚才我们分析的`cpu_exec()`触发的，由一整套 TCG 翻译路径完成的。

我们询问大模型qemu进行指令翻译的具体逻辑，总结如下

指令翻译的总体流程为

`cpu_exec()`

└─ `cpu_exec_loop()`

└─ `tb = tb_find()`

└─ 若 TB 不存在

└─ `tb_gen_code()` ← 【翻译入口】

```
|      └─ translator_loop()
|      └─ riscv_tr_translate_insn()
|      └─ decode + tcg_gen_xxx()
└─ tcg_qemu_tb_exec() ← 执行翻译后的 TB
```

其中 `tb_gen_code()` 函数是指令翻译的翻译入口，它定义在文件 `accel/tcg/translate-all.c` 中，主要作用是创建一个新的 Translation Block、初始化 TCG 上下文和调用架构相关的翻译循环。

根据以上相关分析，我们在 **qemu** 中设置如下断点

```
b tb_gen_code    //翻译是否被触发
b translator_loop //进入翻译主循环
b riscv_tr_translate_insn //RISC-V 指令级翻译
b tcg_gen_ld_i64  //TCG IR 生成（例如访存）
```

结果如下所示

```
(gdb) b tb_gen_code
Breakpoint 9 at 0x5c9908d0daec: file /home/osaq/qemu-4.1.1-debug/accel/tcg/translate-all.c, line 1673.
(gdb) b translator_loop
Breakpoint 10 at 0x5c9908d0f5c2: file /home/osaq/qemu-4.1.1-debug/accel/tcg/translator.c, line 35.
(gdb) b riscv_translate_insn
Function "riscv_translate_insn" not defined.
Make breakpoint pending on future shared library load? (y or [n]) n
(gdb) b tcg_gen_ld_i64
Breakpoint 11 at 0x5c9908c971ab: tcg_gen_ld_i64. (2 locations)
(gdb) b riscv_tr_translate_insn
Breakpoint 12 at 0x5c9908d9b56c: file /home/osaq/qemu-4.1.1-debug/target/riscv/translate.c, line 795.
(gdb) c
Continuing.
```

接着运行代码，可以看到qemu程序确实停在了我设置的函数断点，整体大致过程如下所示


```

Thread 3 "qemu-system-ris" hit Breakpoint 9, tb_gen_code (
  cpu=0xffffffff00000000, pc=18376938279485333505,
  cs_base=2097152, flags=23705, cflags=994951072)
  at /home/osaq/qemu-4.1.1-debug/accel/tcg/translate-all.c:1673
1673 {
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 10, translator_loop (
  ops=0x77fc5c000eca, db=0x5c9908ce2e32 <tlb_index+32>,
  cpu=0x5c9908c6bd7f <qemu_ram_addr_from_host+52>,
  tb=0x77fc712807e0, max_insns=1986797089)
  at /home/osaq/qemu-4.1.1-debug/accel/tcg/translator.c:35
35 {
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 12, riscv_tr_translate_in
sn (dcbase=0x5c9908d9b4f9 <riscv_tr_insn_start+52>,
  cpu=0x77fc71280780)
  at /home/osaq/qemu-4.1.1-debug/target/riscv/translate.c:795
795 {
(gdb) c
Continuing.

```

接着我们来使用单步调试深入分析这里调用的函数的内部处理逻辑，具体如下所示

```

Thread 3 "qemu-system-ris" hit Breakpoint 9, tb_gen_code (
  cpu=0xffffffff00000000, pc=18376938279485333505,
  cs_base=2097152, flags=23705, cflags=994951072)
  at /home/osaq/qemu-4.1.1-debug/accel/tcg/translate-all.c:1673
1673 {
(gdb) n
1674     CPUArchState *env = cpu->env_ptr;
(gdb) n
1686     phys_pc = get_page_addr_code(env, pc);
(gdb) n
1688     if (phys_pc == -1) {
(gdb) n
1694         cflags &= ~CF_CLUSTER_MASK;
(gdb) n
1695         cflags |= cpu->cluster_index << CF_CLUSTER_SHIFT;
(gdb) n
1697         max_insns = cflags & CF_COUNT_MASK;
(gdb) n
1698         if (max_insns == 0) {
(gdb) n
1699             max_insns = CF_COUNT_MASK;
(gdb) n
1701         if (max_insns > TCG_MAX_INSNS) {
(gdb) n
1702             max_insns = TCG_MAX_INSNS;
(gdb) n
1704         if (cpu->singlestep_enabled || singlestep) {
(gdb) n
1705             max_insns = 1;
(gdb) n
1709         tb = tb_alloc(pc);
(gdb) n
1710         if (unlikely(!tb)) {

```

```

1719     gen_code_buf = tcg_ctx->code_gen_ptr;
(gdb) n
1720     tb->tc_ptr = gen_code_buf;
(gdb) n
1721     tb->pc = pc;
(gdb) n
1722     tb->cs_base = cs_base;
(gdb) n
1723     tb->flags = flags;
(gdb) n
1724     tb->cflags = cflags;
(gdb) n
1725     tb->trace_vcpu_dstate = *cpu->trace_dstate;
(gdb) n
1726     tcg_ctx->tb_cflags = cflags;
(gdb) n
1735     tcg_func_start(tcg_ctx);
(gdb) n
1737     tcg_ctx->cpu = env_cpu(env);
(gdb) n
1738     gen_intermediate_code(cpu, tb, max_insns);
(gdb) n

Thread 3 "qemu-system-ris" hit Breakpoint 10, translator_loop (
    ops=0x77fc5c000ec8, db=0x5c9908ce2e32 <tlb_index+32>,
    cpu=0x5c9908c6bd7f <qemu_ram_addr_from_host+52>,
    tb=0x77fc712807e0, max_insns=1986797089)
    at /home/osaq/qemu-4.1.1-debug/accel/tcg/translator.c:35
35     {
(gdb) n
36     int bp_insn = 0;
(gdb) n
39     db->tb = tb;
(gdb) n
40     db->pc_first = tb->pc;
(gdb) n
41     db->pc_next = db->pc_first;

```

```

42         db->is_jump = DISAS_NEXT;
(gdb) n
43         db->num_insns = 0;
(gdb) n
44         db->max_insns = max_insns;
(gdb) n
45         db->singlestep_enabled = cpu->singlestep_enabled;
(gdb) n
47         ops->init_disas_context(db, cpu);
(gdb) n
48         tcg_debug_assert(db->is_jump == DISAS_NEXT); /* no earl
y exit */
(gdb) n
51         tcg_clear_temp_count();
(gdb) n
54         gen_tb_start(db->tb);
(gdb) n
55         ops->tb_start(db, cpu);
(gdb) n
56         tcg_debug_assert(db->is_jump == DISAS_NEXT); /* no earl
y exit */
(gdb) n
59         db->num_insns++;
(gdb) n
60         ops->insn_start(db, cpu);
(gdb) n
61         tcg_debug_assert(db->is_jump == DISAS_NEXT); /* no
early exit */
(gdb) n
64         if (!db->singlestep_enabled
(gdb) n
88         if (db->num_insns == db->max_insns
(gdb) n
89             && (tb_cflags(db->tb) & CF_LAST_IO)) {
(gdb) n
95         ops->translate_insn(db, cpu);
(gdb) n

```

```

Thread 3 "qemu-system-ris" hit Breakpoint 12, riscv_tr_translate_in
sn (dcbase=0x5c9908d9b4f9 <riscv_tr_insn_start+52>,
  cpu=0x77fc71280780)
  at /home/osaq/qemu-4.1.1-debug/target/riscv/translate.c:795
795     {
(gdb) n
796         DisasContext *ctx = container_of(dcbase, DisasContext,
base);
(gdb) n
797         CPURISCVState *env = cpu->env_ptr;
(gdb) n
799         ctx->opcode = cpu_ldl_code(env, ctx->base.pc_next);
(gdb) n
800         decode_opc(ctx);
(gdb) n
801         ctx->base.pc_next = ctx->pc_succ_insn;
(gdb) n
803         if (ctx->base.is_jump == DISAS_NEXT) {
(gdb) n
806             page_start = ctx->base.pc_first & TARGET_PAGE_MASK;
(gdb) n
807             if (ctx->base.pc_next - page_start >= TARGET_PAGE_S
IZE) {
(gdb) n
811         }
(gdb) n
translator_loop (ops=0x5c99095eb360 <riscv_tr_ops>,
  db=0x77fc71280800, cpu=0x5c993b4d3590,
  tb=0x77fc6a000040 <code_gen_buffer+19>, max_insns=1)
  at /home/osaq/qemu-4.1.1-debug/accel/tcg/translator.c:99
99         if (db->is_jump != DISAS_NEXT) {
(gdb) n
105         if (tcg_op_buf_full() || db->num_insns >= db->max_i
nsns) {
(gdb) n
106             db->is_jump = DISAS_TOO_MANY;

```

以上即为我们通过gdb跟踪得到的基本较为完整的QEMU的TCG指令翻译主路径，整理如下，主要分为三层

第一层：tb_gen_code() —— TB的“创建与配置阶段”

```

tb_gen_code()

1  tb_gen_code (
2  cpu=0xfffffffff0000000, pc=1837693827948533505,
3  cs_base=2907152, flags=23705, cflags=994951072)
4  {
5      CPUArchState *env = cpu->env_ptr;
6      phys_pc = get_page_addr_code(env, pc);
7      if (phys_pc == -1) {
8          cflags &= ~CF_CLUSTER_MASK;
9          cflags |= cpu->cluster_index << CF_CLUSTER_SHIFT;
10         max_insns = cflags & CF_COUNT_MASK;
11         if (max_insns == 0) {
12             max_insns = CF_COUNT_MASK;
13         if (max_insns > TCG_MAX_INSNS) {
14             max_insns = TCG_MAX_INSNS;
15         if (cpu->singlestep_enabled || singlestep) {
16             max_insns = 1;

```

```

17         tb = tb_alloc(pc);
18         if (unlikely(!tb)) {
19             gen_code_buf = tcg_ctx->code_gen_ptr;
20             tb->tc_ptr = gen_code_buf;
21             tb->pc = pc;
22             tb->cs_base = cs_base;
23             tb->flags = flags;
24             tb->cflags = cflags;
25             tb->trace_vcpu_dstate = *cpu->trace_dstate;
26             tcg_ctx->tb_cflags = cflags;
27             tcg_func_start(tcg_ctx);
28             tcg_ctx->cpu = env_cpu(env);
29             gen_intermediate_code(cpu, tb, max_insns);

```

tb_gen_code是整个翻译流程的入口函数，执行逻辑如下

1、建立 CPU 架构上下文

```
CPUArchState *env = cpu->env_ptr;
```

其中 env = RISC-V 的 CPURISCVState，包含 GPR / CSR、特权级、satp / mstatus 等，后面所有取指、异常判断都用它

2、PC → 物理地址

```
phys_pc = get_page_addr_code(env, pc);
```

这是第一次重要分叉点：

- 成功：代码页可执行
- 失败：phys_pc == -1
 - 代码页不存在 / 无权限
 - 触发异常（取指缺页 / 指令访问异常）

我们这里：

```
1688 if (phys_pc == -1) {
```

没有进 if，说明：取指地址合法，可翻译（注意：翻译期也会走 MMU / TLB!）

3、计算 TB 的翻译边界（max_insns）

```

max_insns = cflags & CF_COUNT_MASK;
...
if (cpu->singlestep_enabled || singlestep)
    max_insns = 1;

```

其实我们在后面也再次看到了

translator_loop(..., max_insns=1)

说明当前 TB 只允许翻译 1 条指令，可能是由于单步调试、I/O 边界、CF_LAST_IO等原因。

4、分配 TB 并写入元信息

```
tb = tb_alloc(pc);
tb->pc = pc;
tb->cs_base = cs_base;
tb->flags = flags;
tb->cflags = cflags;
```

此时 TB 只是一个“空壳”，它还没有任何指令，也没有 host 代码，只是一个翻译容器。

5、初始化 TCG 函数

```
tcg_func_start(tcg_ctx);
tcg_ctx->cpu = env_cpu(env);
```

这一步非常重要，主要是清空 TCG IR，之后生成的所有 TCG op 都写进当前 TB。

6、进入真正的翻译循环

```
gen_intermediate_code(cpu, tb, max_insns);
```

在这个函数之后，开始真正开始调用 RISC-V 的翻译代码，把 RISC-V 指令变成 TCG IR。

`gen_intermediate_code()` 是通用翻译框架与具体 CPU 架构翻译逻辑的分界点，在此之后，翻译过程完全由架构相关的 `TranslatorOps` 驱动，完成指令取值、译码与语义生成。

第二层： `translator_loop()` —— “逐条指令翻译” 的核心循环

```
translator_loop()

1  translator_loop (
2  ops=0x77fc5c00ec8, db=0x5c998ce2e32 <<tlb_index+32>,
3  cpu=0x5c998c6bd7f <qemu_ram_addr_from_host+52>,
4  tb=0x77fc712807e0, max_insns=1986797089)
5  {
6      int bp
7      db->is_jump = DISAS_NEXT;
8      db->num_insns = 0;
9      db->max_insns = max_insns;
10     db->singlestep_enabled = cpu->singlestep_enabled;
11     ops->init_disas_context(db, cpu);
12     tcg_debug_assert(db->is_jump == DISAS_NEXT); /* no early exit */
13     tcg_clear_temp_count();
14     gen_tb_start(db->tb);
15     ops->tb_start(db, cpu);
16     tcg_debug_assert(db->is_jump == DISAS_NEXT); /* no early exit */
17     db->num_insns++;
18     ops->insn_start(db, cpu);
19     tcg_debug_assert(db->is_jump == DISAS_NEXT); /* no early exit */
20     if (!db->singlestep_enabled
```

```

21         if (db->num_insns == db->max_insns
22             && (tb_cflags(db->tb) & CF_LAST_IO)) {
23             ops->translate_insn(db, cpu);

```

1、初始化 DisasContext (db)

```

db->is_jump = DISAS_NEXT;
db->num_insns = 0;
db->max_insns = max_insns;
db->singlestep_enabled = cpu->singlestep_enabled;

```

db 里保存：

pc_first: TB起始PC

pc_next: 当前指令PC

is_jump: TB是否结束

num_insns: 已翻译条数

2、架构相关初始化

```
ops->init_disas_context(db, cpu);
```

对 RISC-V 来说 ops = riscv_tr_ops，所以实际调用：

```
riscv_tr_init_disas_context()。
```

这里会读特权级、设置 XLEN并且初始化 PC。

3、TB 开始标记

```

gen_tb_start(db->tb);
ops->tb_start(db, cpu);

```

这里插入 TCG 标签，用于异常回退 / 调试

4、正式进入“每条指令”的循环

```

db->num_insns++;
ops->insn_start(db, cpu);
...
ops->translate_insn(db, cpu);

```

在 QEMU 的 TCG 架构中，translate_insn 是架构相关的核心函数，它承担了指令取值、译码以及语义生成的入口职责，是连接通用翻译框架与具体 ISA 行为的关键桥梁。

第三层： riscv_tr_translate_insn() —— 单条 RISC-V 指令的翻译

```
riscv_tr_translate_insn()
```

```

1  riscv_tr_translate_insn
2  (dcbase=0x5c998d9b4f9 <riscv_tr_insn_start+52>,cpu=0x77fc71280780)

```



```

3      {
4          DisasContext *ctx = container_of(dcbase, DisasContext, base);
5          CPURISCVState *env = cpu->env_ptr;
6          ctx->opcode = cpu_ldl_code(env, ctx->base.pc_next);
7          decode_opc(ctx);
8          ctx->base.pc_next = ctx->pc_succ_insn;
9          if (ctx->base.is_jmp == DISAS_NEXT) {
10             page_start = ctx->base.pc_first & TARGET_PAGE_MASK;
11             if (ctx->base.pc_next - page_start >= TARGET_PAGE_SIZE) {
12

```

1、取 guest 指令（翻译期）

```
ctx->opcode = cpu_ldl_code(env, ctx->base.pc_next);
```

这一步非常关键，是**翻译期取指**，使用 `cpu_ldl_code` 函数，读取当前 PC 指向的 guest 指令。该取指操作并不是简单的内存读取，而是**完整地经过 QEMU 的访存机制，包括 TLB 查询、页表转换以及执行权限检查**。这意味着，即使是在生成 TB 的过程中，QEMU 仍然严格遵循 guest 架构的内存保护规则。如果在该步骤中发生**取指异常**（如页不存在或无执行权限），QEMU 会通过异常路径直接跳转到异常处理流程，当前 TB 的生成过程被中断，且不会生成任何可执行的翻译块。

2、指令译码

```
decode_opc(ctx);
```

成功取到指令后，QEMU 调用 `decode_opc(ctx)` 对 opcode 进行译码。该过程根据指令的二进制编码解析其类型和操作数格式，并据此确定后续的翻译函数。对于 RISC-V 架构而言，译码阶段会将不同的指令分派到对应的 `trans_xxx()` 函数，例如算术指令、访存指令、特权指令（如 `ecall`、`sret`）等。译码阶段是架构相关逻辑的集中体现，它决定了指令语义由哪一段翻译代码来生成。

3、计算下一条 PC

```
ctx->base.pc_next = ctx->pc_succ_insn;
```

在译码完成后，QEMU 会更新下一条指令的程序计数器，即

```
ctx->base.pc_next = ctx->pc_succ_insn;。
```

其中，`pc_succ_insn` 通常在译码阶段已经计算完成。对于普通的 32 位 RISC-V 指令，下一条 PC 为当前 PC 加 4；对于压缩指令（RVC），下一条 PC 为当前 PC 加 2。若指令属于跳转或分支指令，其实际的下一条 PC 可能会在对应的 `trans_xxx()` 函数中被重新修改。该步骤为后续是否继续在同一个 TB 中翻译指令提供依据。

4、跨页检查（TB 终止条件）

```

page_start = ctx->base.pc_first & TARGET_PAGE_MASK;
if (ctx->base.pc_next - page_start >= TARGET_PAGE_SIZE)

```

在更新 PC 之后，QEMU 会检查下一条指令是否仍位于当前内存页内，即通过比较 `pc_next` 与 TB 起始地址所在页的边界。如果发现下一条指令将跨越页边界，则当前 TB 的翻译过程会被终止。QEMU

这样设计的原因在于，不同内存页可能具有不同的 MMU 映射或访问权限，同时 TLB 项也是以页为粒度进行管理的。禁止 TB 跨页可以简化权限检查与异常处理逻辑，确保翻译块内部的执行环境在整个 TB 生命周期内保持一致。

由于这里没有涉及到访存指令，没有看到 `b tcg_gen_ld_i64` 调用的情况（我们尝试将其他三个断点设置为 `disable`，运行 `c` 发现终端 3 直接进入了下一个中断，说明这部分没有涉及到访存指令）

调试过程中，还看到了这两段代码

```
gen_intermediate_code (cs=0x5c993b4d3590,
    tb=0x77fc6a000040 <code_gen_buffer+19>, max_insns=1)
    at /home/osaq/qemu-4.1.1-debug/target/riscv/translate.c:849
849     }
(gdb) n
tb_gen_code (cpu=0x5c993b4d3590, pc=18446744072637910728,
    cs_base=0, flags=24577, cflags=-16252928)
    at /home/osaq/qemu-4.1.1-debug/accel/tcg/translate-all.c:1739
1739     tcg_ctx->cpu = NULL;
(gdb) n
1741     trace_translate_block(tb, tb->pc, tb->tc.ptr);
(gdb) c
Continuing.
```

询问大模型分析可知这是 QEMU TCG 翻译流程的收尾阶段：首先，负责 RISC-V 中间代码生成的 `gen_intermediate_code` 函数执行完毕并退出；随后流程回到 `tb_gen_code`，先清空 TCG 上下文与 CPU 的关联（即设置 `tcg_ctx->cpu = NULL`），再调用 `trace_translate_block` 记录下翻译块的地址、程序计数器等信息；最后在 GDB 中输入 `c`（即 `continue`）指令，让程序继续运行。整个流程完成了单条 RISC-V 指令的翻译工作，同时也为翻译块的生成完成了收尾操作。

注：下面这两个要求关键部分都放在调试过程中了，不再这里单独列出来了。

记录下你调试过程中比较抓马有趣的细节，以及在观察模拟器通过软件模拟硬件执行的时候了解到的知识。

记录实验过程中，有哪些通过大模型解决的问题，记录下当时的情景，你的思路，以及你和大模型交互的过程。